# SQL-də Soft Delete

*Nəzəri və Praktiki Bələdçi*

Verilənlər Bazası İdarəetmə Sistemləri
Tarix: 3 Yanvar 2026

## 1. Giriş

**Soft Delete** (Yumşaq Silinmə) - məlumatın verilənlər bazasından fiziki olaraq silinməyib, ancaq silinmiş kimi qeyd edilməsi üsuludur. Bu metodologiya müasir proqram təminatı sistemlərində geniş istifadə olunur və məlumat təhlükəsizliyi, audit və geri qaytarma imkanları təmin edir.

> **Əsas Prinsip:** Məlumat əslində verilənlər bazasında qalır, sadəcə xüsusi işarə (marker) ilə "silinmiş" statusu alır və standart sorğularda görünmür.

## 2. Hard Delete və Soft Delete Müqayisəsi

| Xüsusiyyət | Hard Delete | Soft Delete |
|---|---|---|
| **Əməliyyat** | DELETE FROM table | UPDATE table SET deleted_at |
| **Məlumat** | Tamamilə silinir | Bazada qalır |
| **Geri qaytarma** | Qeyri-mümkün | Mümkün |
| **Performans** | CASCADE əməliyyatlar ağır | Sadə UPDATE |
| **Audit** | Tarixçə itirilir | Tam tarixçə |
| **Baza həcmi** | Azalır | Artır |

### 2.1. Hard Delete Nümunəsi

```
-- İstifadəçini tamamilə sil
DELETE FROM users WHERE id = 5;

-- Nəticə: Məlumat bazadan yox olur, geri qaytarıla bilməz
```

### 2.2. Soft Delete Nümunəsi

```
-- İstifadəçini soft delete et
UPDATE users SET deleted_at =
CURRENT_TIMESTAMP,     deleted_by =
1 WHERE id = 5;

-- Nəticə: Məlumat bazada qalır, ancaq "silinmiş" kimi işarələnir
```

## 3. İstifadə Səbəbləri

### 3.1. Məlumat Təhlükəsizliyi və Bərpa

İstifadəçilərin və ya sistemin etdiyi səhv əməliyyatları geri qaytarmaq imkanı. Məsələn, istifadəçi səhvən vacib məlumatı silsə, sistem administratoru həmin məlumatı asanlıqla bərpa edə bilər.

### 3.2. Audit və Tarixçə (Audit Trail)

Hüquqi və korporativ tələblər üçün vacibdir. Kim, nə vaxt, hansı məlumatı sildi - bütün bu informasiya saxlanılır və lazım olduqda təqdim edilə bilər.

### 3.3. Referensial Bütövlük

Əlaqəli məlumatların strukturunu pozmadan saxlamaq. Məsələn, istifadəçi silinərkən onun etdiyi sifarişlər, yazılar və digər əlaqəli məlumatlar qorunur.

### 3.4. İş Prosesləri

Mühasibatlıq, tibb, maliyyə sistemlərində məlumatın "silinməsi" əvəzinə "arxivləşdirilməsi" tələb olunur. Qanunvericilik bəzi məlumatların müəyyən müddət saxlanılmasını tələb edir.

### 3.5. Performans

CASCADE DELETE əməliyyatları çoxlu əlaqəli cədvəllərə toxunduğu üçün çox vaxt aparır. Soft delete sadəcə bir UPDATE olduğu üçün daha sürətlidir.

## 4. Nəzəri Əsaslar

### 4.1. Tətbiq Üsulları

#### 4.1.1. Boolean Flag (Ən Sadə Yanaşma)

```
-- Cədvələ sütun əlavə et
ALTER TABLE users ADD COLUMN is_deleted BOOLEAN DEFAULT FALSE;

-- İstifadə
UPDATE users SET is_deleted = TRUE WHERE id = 5;
SELECT * FROM users WHERE is_deleted = FALSE;
```

**Məhdudiyyət:** Silinmə tarixi və kim tərəfindən silindiyini bilmirik.

#### 4.1.2. Timestamp (Daha İnformativ)

```
-- Cədvələ sütun əlavə et
ALTER TABLE users ADD COLUMN deleted_at TIMESTAMP NULL;

-- İstifadə
UPDATE users SET deleted_at = CURRENT_TIMESTAMP WHERE id = 5;
SELECT * FROM users WHERE deleted_at IS NULL;
```

**Üstünlük:** Nə vaxt silindiyini bilirik, daha çevik sorğular yazmaq mümkündür.

#### 4.1.3. Tam Audit (Tövsiyə Olunan)

```
-- Cədvələ sütunlar əlavə et
ALTER TABLE users ADD COLUMN deleted_at TIMESTAMP NULL;
ALTER TABLE users ADD COLUMN deleted_by INTEGER REFERENCES users(id);

-- İstifadə
UPDATE users SET deleted_at =
CURRENT_TIMESTAMP,     deleted_by =
1 WHERE id = 5;
```

**Üstünlük:** Tam audit trail - nə vaxt və kim sildi.

### 4.2. İndeksləşdirmə Strategiyası

Performans üçün çox vacibdir. Silinməmiş məlumatlar üçün partial index yaratmalıyıq:

```
-- Aktiv istifadəçilər üçün index CREATE
INDEX idx_active_users
ON users(id)
WHERE deleted_at IS NULL;

-- Bu index yalnız deleted_at IS NULL olan sətirləri əhatə edir
-- və sorğuları əhəmiyyətli dərəcədə sürətləndirir
```

## 5. Praktiki Tətbiq

## 5.1. Cədvəl Strukturunun Yaradılması

```
-- İstifadəçilər cədvəli CREATE TABLE users (     id
SERIAL PRIMARY KEY,     username VARCHAR(100) NOT
NULL,     email VARCHAR(255) UNIQUE NOT NULL,
password_hash VARCHAR(255) NOT NULL,     created_at
TIMESTAMP DEFAULT CURRENT_TIMESTAMP,     updated_at
TIMESTAMP DEFAULT CURRENT_TIMESTAMP,     deleted_at
TIMESTAMP NULL,     deleted_by INTEGER REFERENCES
users(id) );

-- Sifarişlər cədvəli CREATE TABLE orders (     id
SERIAL PRIMARY KEY,     user_id INTEGER REFERENCES
users(id),     product_name VARCHAR(255) NOT NULL,
amount DECIMAL(10,2) NOT NULL,     status
VARCHAR(50) DEFAULT 'pending',     created_at
TIMESTAMP DEFAULT CURRENT_TIMESTAMP,     deleted_at
TIMESTAMP NULL );

-- İndekslər
CREATE INDEX idx_active_users ON users(id) WHERE deleted_at IS NULL;
CREATE INDEX idx_active_orders ON orders(id) WHERE deleted_at IS NULL;
CREATE INDEX idx_user_orders ON orders(user_id) WHERE deleted_at IS NULL;
```

## 5.2. CRUD Əməliyyatları

### 5.2.1. CREATE (Yaratma)

Yaratma əməliyyatı dəyişmir, adi INSERT istifadə olunur:

```
INSERT INTO users (username, email, password_hash)
VALUES ('ali_mammadov', 'ali@example.com', 'hashed_password_123');
```

### 5.2.2. READ (Oxuma)

Oxuma əməliyyatlarında mühüm dəyişiklik var - həmişə WHERE deleted_at IS NULL şərti əlavə etməliyik:

```
-- Yalnız aktiv istifadəçilər
SELECT * FROM users WHERE deleted_at IS NULL;

-- Bütün istifadəçilər (admin panel üçün)
SELECT * FROM users;

-- Silinmiş istifadəçilər
SELECT * FROM users WHERE deleted_at IS NOT NULL;

-- Son 7 gündə silinmiş istifadəçilər SELECT
username, email, deleted_at
FROM users WHERE deleted_at > CURRENT_TIMESTAMP -
INTERVAL '7 days'
ORDER BY deleted_at DESC;
```

### 5.2.3. UPDATE (Yeniləmə)

Yeniləmə zamanı yalnız aktiv məlumatları yeniləməliyik:

```
-- Email yenilə (yalnız aktiv istifadəçi üçün)
UPDATE users SET email =
'yeni_email@example.com',
updated_at = CURRENT_TIMESTAMP WHERE
id = 5 AND deleted_at IS NULL;

-- Şifrə dəyişdir
UPDATE users SET password_hash =
'new_hashed_password',     updated_at =
CURRENT_TIMESTAMP WHERE id = 5 AND
deleted_at IS NULL;
```

### 5.2.4. DELETE (Soft Delete)

DELETE əvəzinə UPDATE istifadə edirik:

```
-- Sadə soft delete
UPDATE users SET deleted_at =
CURRENT_TIMESTAMP
WHERE id = 5;

-- Tam audit ilə soft delete
UPDATE users SET deleted_at = CURRENT_TIMESTAMP,
deleted_by = 1  -- silən istifadəçinin ID-si
WHERE id = 5 AND deleted_at IS NULL;

-- Şərtli soft delete (yalnız aktiv olarsa)
UPDATE users SET deleted_at =
CURRENT_TIMESTAMP,     deleted_by =
1 WHERE id = 5
  AND deleted_at IS NULL
  AND status = 'active';
```

## 5.3. Geri Qaytarma (Restore)

```
-- Sadə restore
UPDATE users SET
deleted_at = NULL,
deleted_by = NULL WHERE
id = 5;

-- Audit ilə restore
UPDATE users SET deleted_at = NULL,
deleted_by = NULL,     restored_at =
CURRENT_TIMESTAMP,     restored_by =
2 WHERE id = 5;
```

## 5.4. VIEW-lar (İşi Asanlaşdırma)

Hər dəfə WHERE deleted_at IS NULL yazmamaq üçün VIEW-lar yaratmaq məsləhətdir:

```
-- Aktiv istifadəçilər view
CREATE VIEW active_users AS
SELECT id, username, email, created_at, updated_at
FROM users
WHERE deleted_at IS NULL;

-- Aktiv sifarişlər view
CREATE VIEW active_orders AS SELECT o.id, o.user_id,
u.username, o.product_name, o.amount
FROM orders o
JOIN users u ON o.user_id = u.id
WHERE o.deleted_at IS NULL AND u.deleted_at IS NULL;

-- İndi sadə sorğular:
SELECT * FROM active_users;
SELECT * FROM active_orders WHERE user_id = 5;
```

## 5.5. Trigger-lər (Avtomatlaşdırma)

İstifadəçi silinəndə onun sifarişlərini də avtomatik soft delete etmək:

```
-- Trigger funksiyası CREATE OR REPLACE FUNCTION
soft_delete_user_orders() RETURNS TRIGGER AS $$
BEGIN
    -- İstifadəçi soft delete ediləndə
    IF NEW.deleted_at IS NOT NULL AND OLD.deleted_at IS NULL THEN
        -- Onun bütün sifarişlərini də soft delete et
        UPDATE orders
        SET deleted_at = NEW.deleted_at
        WHERE user_id = NEW.id
          AND deleted_at IS NULL;
    END IF;

    -- İstifadəçi restore ediləndə
    IF NEW.deleted_at IS NULL AND OLD.deleted_at IS NOT NULL THEN
        -- Onun sifarişlərini də restore et
        UPDATE orders
        SET deleted_at = NULL
        WHERE user_id = NEW.id
AND deleted_at = OLD.deleted_at;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger yaradılması CREATE TRIGGER
trigger_soft_delete_orders
AFTER UPDATE ON users
FOR EACH ROW
EXECUTE FUNCTION soft_delete_user_orders();
```

## 5.6. Praktiki Sorğu Nümunələri

```
-- 1. Son 30 gündə silinmiş istifadəçilər SELECT
username, email, deleted_at, deleted_by
FROM users WHERE deleted_at > CURRENT_TIMESTAMP - INTERVAL
'30 days'
ORDER BY deleted_at DESC;

-- 2. Hansı istifadəçi ən çox silinmə əməliyyatı edib? SELECT
    u.username,     COUNT(*)
as deleted_count
FROM users u1
JOIN users u2 ON u1.id = u2.deleted_by
WHERE u2.deleted_at IS NOT NULL
GROUP BY u.username
ORDER BY deleted_count DESC;

-- 3. Aktiv istifadəçilərin aktiv sifarişləri SELECT
    u.username,
    o.product_name,
    o.amount,
    o.created_at
FROM users u
JOIN orders o ON u.id = o.user_id
WHERE u.deleted_at IS NULL
  AND o.deleted_at IS NULL
ORDER BY o.created_at DESC;

-- 4. İstifadəçi statistikası (aktiv və silinmiş)
SELECT
    COUNT(*) FILTER (WHERE deleted_at IS NULL) as active_users,
COUNT(*) FILTER (WHERE deleted_at IS NOT NULL) as deleted_users,
    COUNT(*) as total_users
FROM users;

-- 5. Ən çox silinən məhsullar
SELECT     product_name,
COUNT(*) as delete_count
FROM orders
WHERE deleted_at IS NOT NULL
GROUP BY product_name ORDER
BY delete_count DESC
LIMIT 10;
```

# 6. Mürəkkəb Ssenarilər

## 6.1. Unique Constraint Problemi

Soft delete istifadə edərkən ən çox rast gəlinən problem - unique constraint ilə bağlıdır:

**Problem:** İstifadəçi silindikdən sonra eyni email ilə yeni istifadəçi yaratmaq olmur.

```
-- Səhv yanaşma (işləməyəcək) CREATE
TABLE users (     id SERIAL PRIMARY KEY,
email VARCHAR(255) UNIQUE NOT NULL,
deleted_at TIMESTAMP NULL );

-- Problem: ali@example.com silindikdən sonra
-- yenidən ali@example.com yaratmaq olmur
```

**Həll Yolları:**

```
-- Həll 1: Partial Unique Index (ən yaxşı) CREATE
UNIQUE INDEX idx_unique_active_email
ON users(email)
WHERE deleted_at IS NULL; -- İndi silinmiş email

yenidən istifadə edilə bilər


-- Həll 2: Email-ə timestamp əlavə et
UPDATE users SET email = email || '_deleted_' || EXTRACT(EPOCH FROM
deleted_at),     deleted_at = CURRENT_TIMESTAMP WHERE id = 5;


-- Həll 3: Xüsusi logic
CREATE UNIQUE INDEX idx_email_deleted
ON users(email, COALESCE(deleted_at, '9999-12-31'));
```

## 6.2. Permanent Delete (Həqiqi Silinmə)

Bəzən köhnə məlumatları tamamilə silmək lazımdır (məsələn, GDPR tələbləri):

```
-- 1 ildən köhnə soft deleted məlumatları sil
DELETE FROM users WHERE deleted_at < CURRENT_TIMESTAMP -
INTERVAL '1 year';

-- Və ya əvvəlcə arxivə köçür
INSERT INTO users_archive
SELECT * FROM users
WHERE deleted_at < CURRENT_TIMESTAMP - INTERVAL '1 year';

DELETE FROM users
WHERE deleted_at < CURRENT_TIMESTAMP - INTERVAL '1 year';

-- Scheduled job yaratmaq (PostgreSQL) CREATE
EXTENSION IF NOT EXISTS pg_cron;

SELECT cron.schedule(
    'cleanup-old-deleted-users',     '0 2 * *
0',  -- Hər bazar günü saat 02:00
    $$DELETE FROM users       WHERE deleted_at < CURRENT_TIMESTAMP
- INTERVAL '1 year'$$
);
```

## 6.3. Stored Procedure (Tam Funksionallıq)

```
-- Kompleks soft delete funksiyası CREATE OR
```

```
REPLACE FUNCTION soft_delete_user(
p_user_id INTEGER,     p_deleted_by INTEGER )
RETURNS TABLE(     success BOOLEAN,     message
TEXT ) AS $$ DECLARE     v_user_exists BOOLEAN;
v_already_deleted BOOLEAN; BEGIN     -- Yoxla:
istifadəçi mövcuddur?
    SELECT EXISTS(         SELECT 1 FROM users
WHERE id = p_user_id
    ) INTO v_user_exists;

    IF NOT v_user_exists THEN         RETURN QUERY SELECT
FALSE, 'İstifadəçi tapılmadı';
        RETURN;
    END IF;
        -- Yoxla: artıq
silinib?
    SELECT EXISTS(
        SELECT 1 FROM users
        WHERE id = p_user_id
          AND deleted_at IS NOT NULL
) INTO v_already_deleted;

    IF v_already_deleted THEN         RETURN QUERY SELECT FALSE,
'İstifadəçi artıq silinib';
        RETURN;
    END IF;

    -- Soft delete əməliyyatı
    UPDATE users     SET deleted_at =
CURRENT_TIMESTAMP,         deleted_by =
p_deleted_by     WHERE id = p_user_id;

    RETURN QUERY SELECT TRUE, 'İstifadəçi uğurla silindi';
END;
$$ LANGUAGE plpgsql; --
```

İstifadə nümunəsi:

```
SELECT * FROM soft_delete_user(5, 1);

-- Restore funksiyası CREATE OR REPLACE
FUNCTION restore_user(     p_user_id
INTEGER,     p_restored_by INTEGER )
RETURNS TABLE(     success BOOLEAN,
message TEXT ) AS $$
BEGIN
    IF NOT EXISTS(
        SELECT 1 FROM users
        WHERE id = p_user_id
AND deleted_at IS NOT NULL
    ) THEN         RETURN QUERY SELECT FALSE, 'İstifadəçi tapılmadı və
ya aktiv';
        RETURN;
    END IF;

    UPDATE users     SET deleted_at =
NULL,         deleted_by = NULL,
restored_at = CURRENT_TIMESTAMP,
restored_by = p_restored_by     WHERE id
= p_user_id;

    RETURN QUERY SELECT TRUE, 'İstifadəçi uğurla bərpa edildi';
END;
$$ LANGUAGE plpgsql;
```

## 7. Üstünlük və Mənfi Tərəflər

### Üstünlüklər

- Məlumat itkisi riski yoxdur
- Geri qaytarma mümkündür
- Tam audit trail mövcuddur
- Referensial əlaqələr qorunur
- GDPR və compliance uyğunluq

### Mənfi Tərəflər

- CASCADE DELETE problemləri yoxdur

  Performans üstünlükləri (bəzi hallarda)
- Tarixçə və analitika üçün ideal
-